



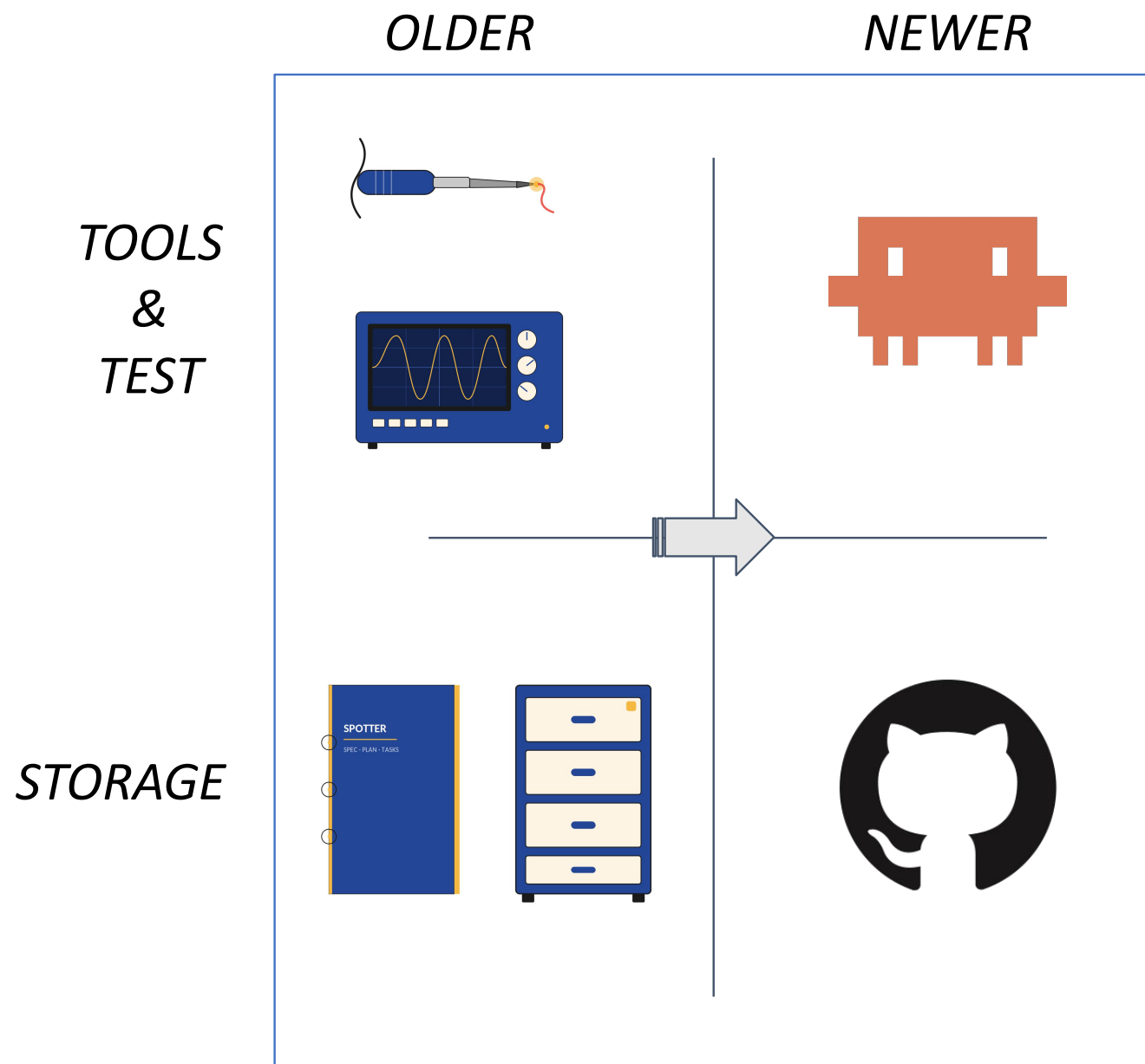
The Spec Is the New Schematic

Reproducible Station Software with Spec-Driven Development (SDD)

John Carter, N6YU — RSGB Convention 2026



■ Software Is Replacing Solder



- Today's shack: seems like as much software as hardware
- Adding a feature: hard in open source, harder in commercial gear, hardest in closed source
- New Tool Stack - OLD + Software Agents, GitHub
- GitHub is the new file cabinet

Using AI for coding has big drawbacks

- **NOT** Repeatable
- **NOT** Structured

Fortunately good enough for radio projects

Key Terms

Agent — AI that edits files, runs code

Claude Code — AI agent in your terminal

CLAUDE.md — Notes Claude reads at startup

Commit — A saved file snapshot with a note

Context window — The AI's working memory

GitHub — Storage website; where files are tracked

Markdown (.md) — Formatted plain text

Masterplan — **Detailed Specs, Tasks... for your app**

Masterplan Generator — **Helper to customize MP**

Push — Upload your files to GitHub

Repo — Project folder plus its history

SDD — AI-updated plan of record

Session — One continuous AI conversation

Slash command — Shortcuts /usage, /status, etc.

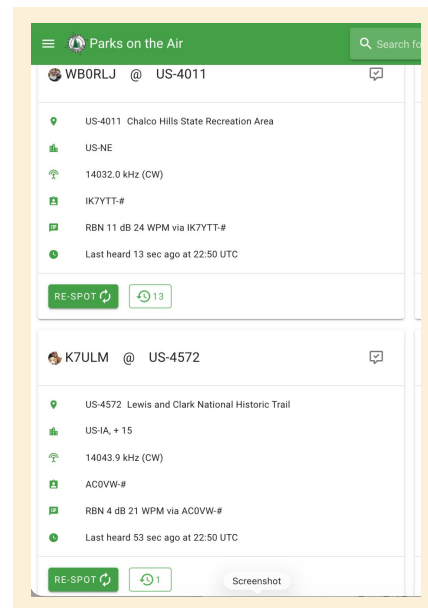
Specs — **Your app's features and behaviors**

Terminal — Text-only app, no mouse

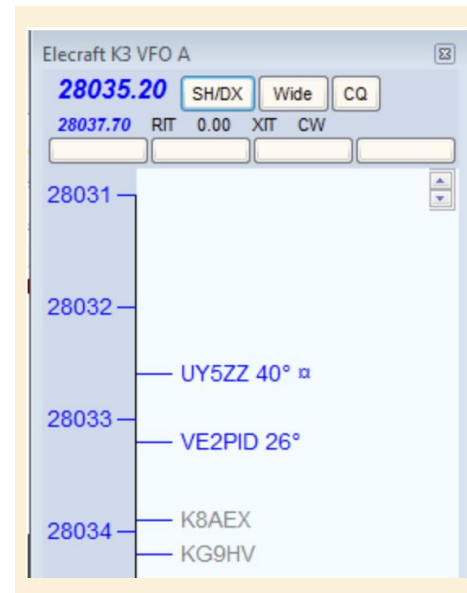
Token — Word-piece used to meter AI use

■ Why Did I Bother? A Poor Man's Panadapter

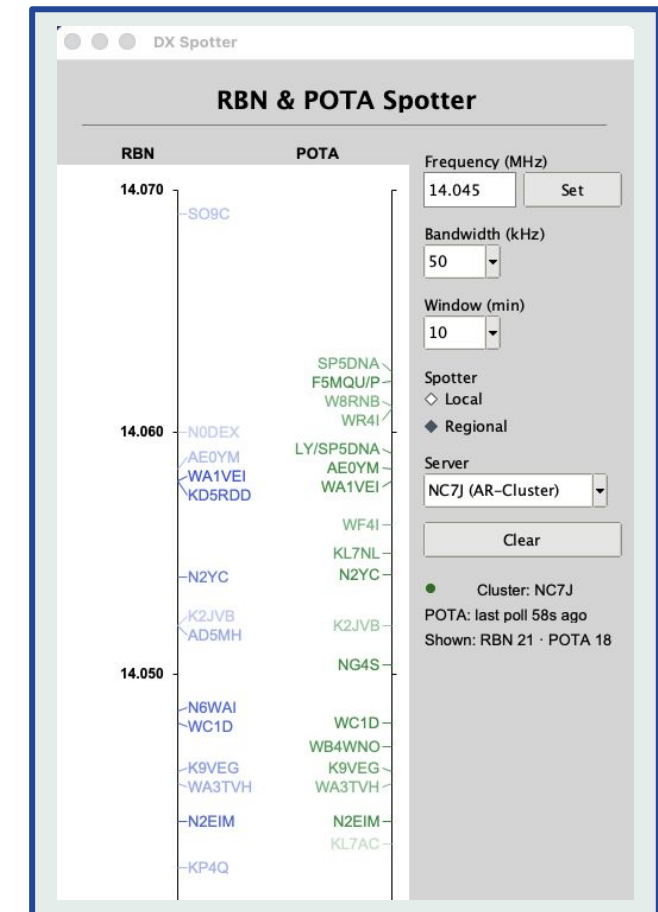
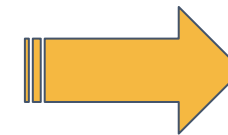
- POTA.APP shows maps, and spots for Hunting
 - Interface prioritizes mapping, not bandscope
- N1MM Windows app shows a bandscope, not POTA spots



POTA.APP



N1MM

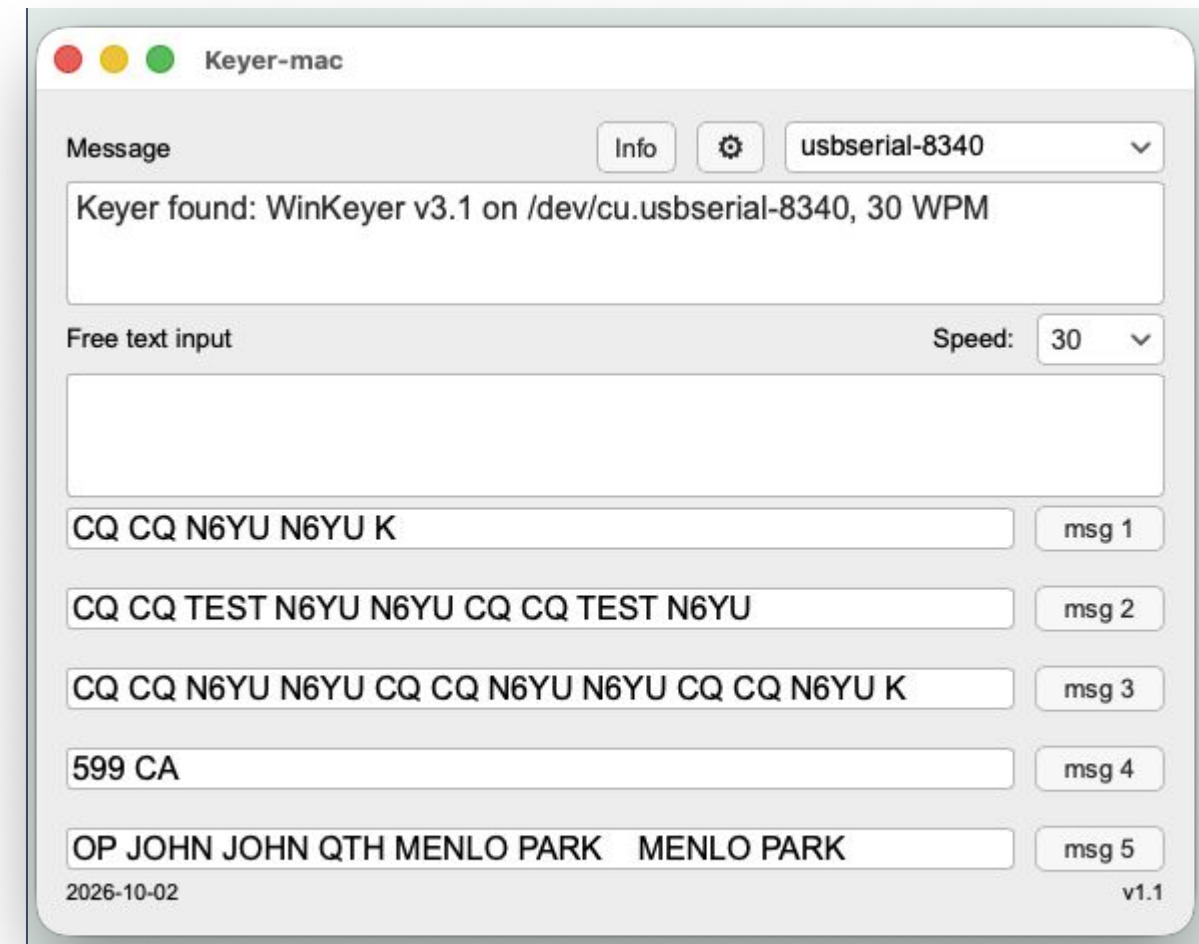


SDD DX Spotter - Integrated

■ Possible SDD Projects for Radio

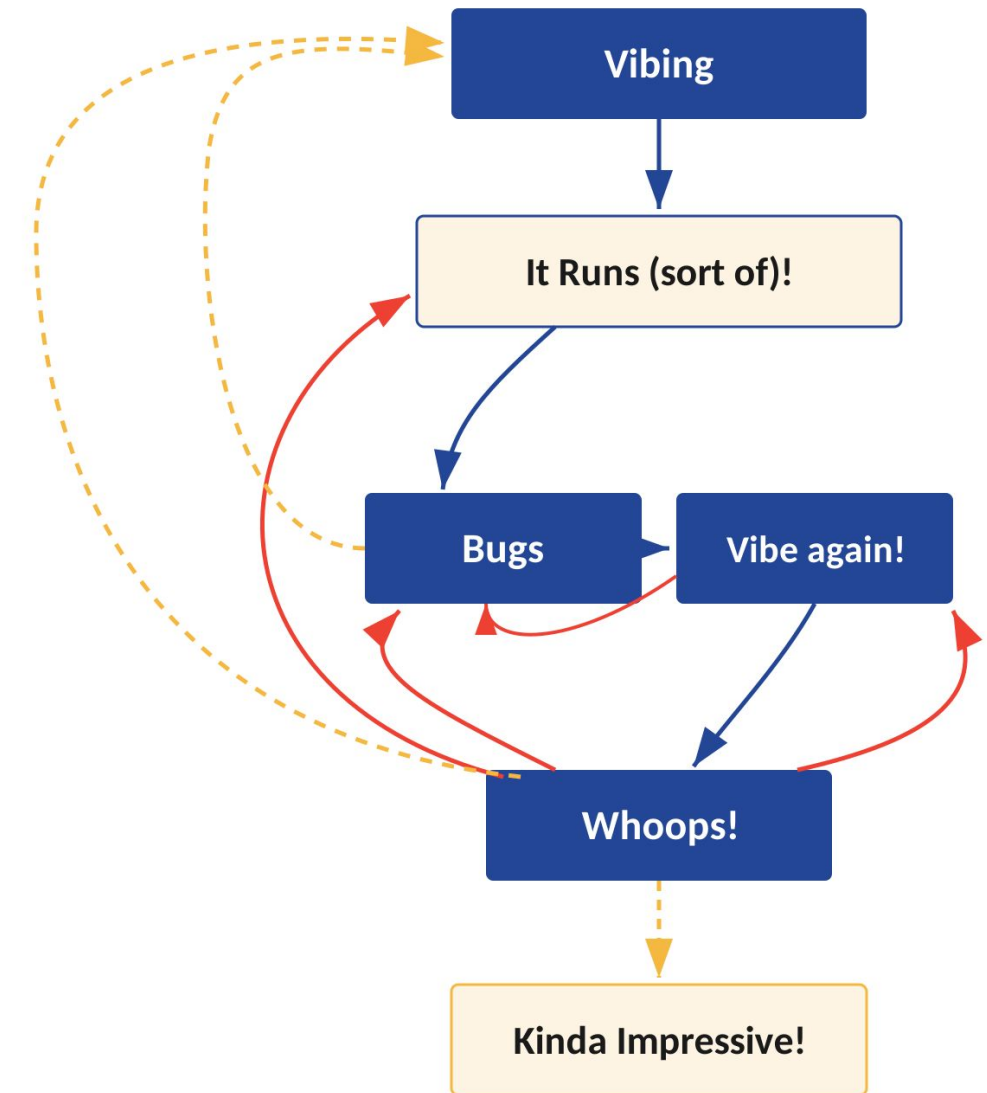
- Platform changes (Win<>Mac<>Linux<>Mobile)
- Open source / GitHub building assistance
- Create code from magazines
- Arduino/Raspberry Pi Projects
- Instrumentation & Automation
 - Data Logging, Temp, Voltage, Current, SWR
- Contact Logging, Tracking Awards, Spotting, Alerts, Band Activity, Antenna Testing
- Station Control: Amps, Switching, Tuners
- Web applications

SDD Example of Keyer built from K6GTE
Platform Change: Linux -> Mac



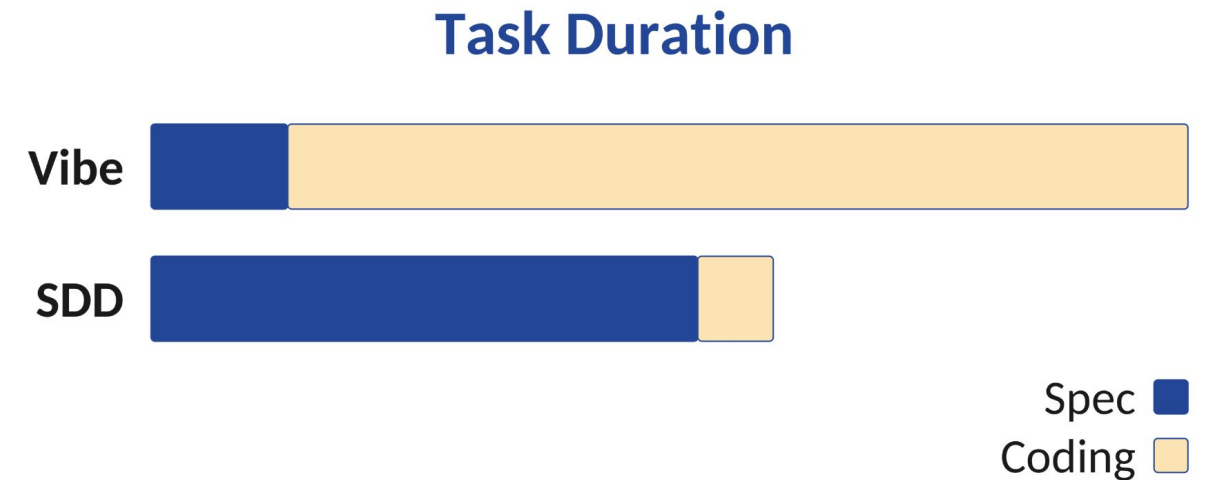
■ Vibe Coding, Can Be Amazing!

- Vague prompt: “create a spotting app for RBN & POTA” — **silent assumptions**
- Functionality: Display, clusters (CC Cluster, DX Spider, AR6?), UI, parameters — **all guessed, often wrong**
- Testing: Feature tests or just smoke test? — **results in brittle code**
- Now what if you want to come back 6 months from now to tweak it? — **where was that chat?**
- Vibe coding is amazing for what it can accomplish in the first 20% of time — **frequently frustrating**



■ A Better Alternative: SDD (Spec-Driven Development)

- SDD Process: Spec > Build > Test > Update Spec
- ‘Vibing’ wasn’t working for large organizations
- Balance consistency and maintainability without impeding ultimate velocity
- Updating the spec more important than updating code
 - Spec replaces source code as deliverable
- Results
 - Better quality & maintainability

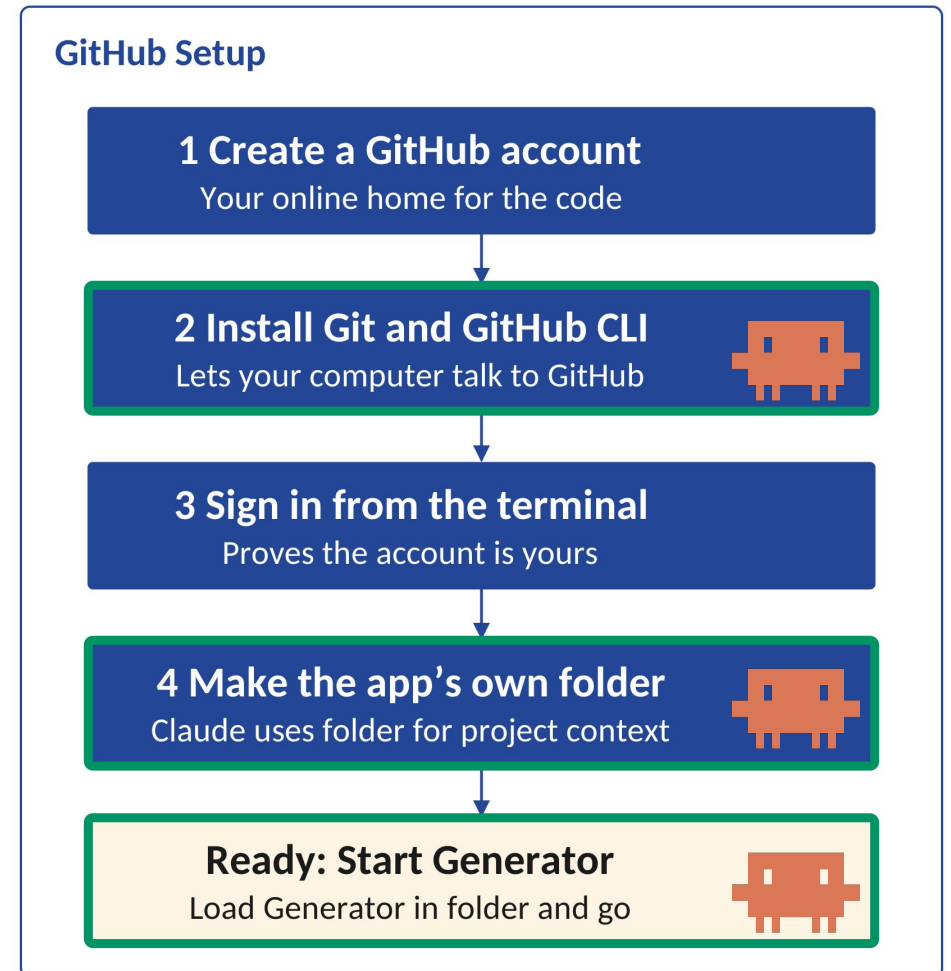


Vibe Prompt vs. SDD

	Vibe coding	Spec-driven
Prompt	“Create a spotting app for RBN & POTA”	idea.md + screenshot & AI to fill in the details
Details	✗ Clusters, UI - guessed, often wrong	✓ Written down, then confirmed or modified and expanded, based on chats with AI
Testing	✗ Smoke test, if any	✓ Each step tested before commit
Speed	✓ Amazing first 20% ✗ Frustrating finish	✗ Slower start; more work up front
Later	✗ “Where is the chat?” ✗ “How can I change X?”	✓ Edit the spec; rebuild from GitHub ✓ Add features or change OS ✓ Repeatable and shareable

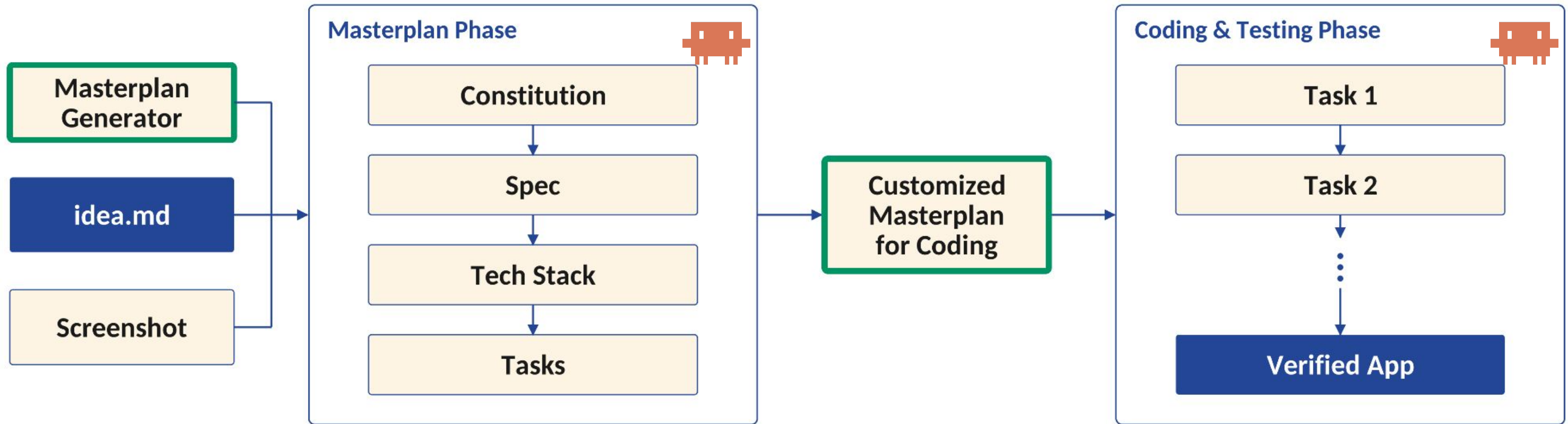
■ Preliminaries: Claude Code & GitHub Setup

- Why Claude Code (and not Claude, or ChatGPT)
 - Code can loop, span agents, execute programs
 - Chat is a text generator, Code does a lot more
- Install and setup
 - Setup GitHub (cloud), gh (local)
 - Authentication needed (can be hassle)
 - Chrome Connectors (Optional, handy)
- GitHub is a Super Power!
 - Claude can create it, sync it, and create Readme file for you, organize it, share it...
 - ...and then you can find it years later



■ Masterplan Generator + Idea + Screenshot

- The Masterplan Generator walks you through the creation of a masterplan.md file tailored to your app
- Cut and paste generator prompts into Claude Code, which helps you create a detailed spec



Use Generator + Your App Idea to Create Detailed Masterplan

Section	Typical Process
1 Constitution how we work	Use the Generator's Constitution prompts as-is: paste into Claude Code, then continue with Spec, Tech, Tasks
2 Spec what it does	Give Claude your app idea + the Generator's Spec questions; have it customize the Spec
3 Tech built with	Give Claude sections 1–2; ask for recommended libraries and approaches
4 Tasks in what order	Ask Claude to structure project into ~5 chunks: Setup, Connections, Core Logic, Main Features, UI (all from Generator)

The Masterplan Generator (Download from Link in Appendix)

masterplan-generator.md

1. Constitution: how we work (*paste all five at once*)

RULE: Build from masterplan.md, idea.md and my screenshot; borrow language, tools, specs, or open-source code from examples as I choose.

RULE: At the start of the build, check tools, libraries, GitHub login, that this folder is its own repo root, and that each data source's host and port can be reached, on my platforms; show pass/fail.

RULE: After each change, measure the app against the Spec's screen list; show pass/fail.

RULE: After each working step: run all tests, show me proof, commit, and push to GitHub. Keep going within a stage; stop only at stage end, on a failed test, after two failed fixes, or for my decision.

RULE: When code and masterplan disagree, propose only major changes, one line each; update the masterplan after I approve.

2. Spec: what the app does (*one at a time*)

DRAFT: Summarize what my app does in a few sentences, from idea.md, my screenshot, and any example app.

DRAFT: From my screenshot, list screen elements and controls, with where each sits.

DRAFT: For each feature in idea.md, describe what the user does and sees, with testable ranges where possible.

DRAFT: List where my app's data comes from, and how we'll check each source works, including after a reconnect. Send every server setting on each connect; never rely on settings from earlier sessions.

USER INPUT: Ask which features from idea.md my app must do this iteration, and which wait; suggest if I'm unsure.

masterplan-generator.md

3. Tech: what it's built with (*one at a time*)

USER INPUT: Recommend a language and tools, Python unless my example suggests better; explain each and let me choose.

DRAFT: List what must be installed on each platform in idea.md, and how to check each works.

RULE: Keep the app responsive while it works, never stuck waiting on data or input.

DRAFT: Propose a handful of well-structured modules, one line each, each testable on its own.

RULE: Keep a short list of known limitations in the masterplan; update it as we go.

4. Tasks: in what order (*one at a time*)

DRAFT: Break the build into about 5 stages: environment (including measuring my screenshot with a script), data connections tested on live servers and ending with a bare window showing live data, core logic, features, and UI. Give each stage sub-steps and one done-when line.

DRAFT: For each sub-step, say how we'll test it and what result means it works.

RULE: Test connections to outside data with real servers before building screens that depend on them.

RULE: Get a simple version running early, then add features one at a time, testing each.

DRAFT: Add a final step: review what went wrong and propose masterplan updates for my approval.

■ Claude Code with Claude Desktop as Helper (Screenshot)

1. Generator causes Code to generate a question I don't understand

4. Paste answer back into Code

Claude Code

dx_spotter/config.py DEFAULT_HOSTS and README.md:

Callsign	Host:port	Software	Status
NC7J	dxc.nc7j.com:7373	AR-Cluster v6.1.5123	Protocol captured live on 2026-09-04 (docs/protocol-notes.md)
WA9PIE-2	dxc.wa9pie.net:8000	DXSpider V1.57 build 686	Wired, verified live
W3LPL	w3lpl.net:7373	DXSpider	Wired
W4MYA	dxc.w4mya.us:7373	CC Cluster 3.397	Skimmer only, verified live

— DXSpider: WA9PIE-2 and W3LPL.
— CC Cluster: W4MYA only.
— Login: the app logs in as your own callsign, N6YU. The W4MYA prompt shows N6YU-0 de W4MYA.
— NC7J: it is AR-Cluster, not CC or DXSpider. I included it because it's the fourth default host.

Neither the README table nor DEFAULT_HOSTS says which of these four is tried first. If you don't set a preferred host, config.py uses the decided order, so NC7J goes first.

I cloned the repo into the session scratchpad, not ~/Projects/, and I didn't change anything in it.

* Churned for 13s · done 3:32 PM

* recap: You wanted to know which cluster servers and callsigns spotter-win2 uses for CC and DXSpider. Answered: DXSpider is WA9PIE-2 and W3LPL, CC Cluster is W4MYA. Next, tell me if you want anything changed or explored further (disable recaps in /config)

✓ Update installed · Restart to update

> Which callsigns did most spots as?

▶ auto mode on (shift+tab to cycle) · ← for agents

Claude Desktop

Reconsidering when and where to filter spot data. >

Approve 1, 2 and 4. Change 3. All four are real issues it found from live data, and it proposed them without editing the masterplan, which is what your Contribution asks for.

- 1, Skimmer match: approve. W6YX-2-# is the real format. Check that the match stays exact on the base call and ignores upper/lower case, with the optional -<digits> and optional -# allowed only at the end.
- 2, POTA spot_id: approve. Without it, POTA spots would never fade, a bug you'd only notice after a few minutes of running. It's a good catch.
- 3, Local/Regional switching: change it. Keep spots from all Regional skimmers in the store, and apply Local at draw time.
 - Local is a subset of Regional, so nothing is lost.
 - Switching then redraws instantly instead of emptying the map.
 - It's the same pattern as pressure-test #4: filter at draw time, not on arrival.
 - Decision 6 changes to match.
- 4, late spots from the old band: approve.

Paste:

Approve 1, 2 and 4. Change 3: the store keeps spots from all Regional skimmers (Local is a subset); the Local/Regional choice applies at draw time, so switching redraws without emptying the map. Update decision 6 to say so. Add all four to masterplan.md, commit and push. Then go for Stage 4.

What can go wrong in Stage 4

- Screen Recording permission: the layout checks capture the app window. The first capture will fail, or come out blank, until you allow your terminal app under System Settings → Privacy & Security → Screen Recording. Then restart terminal.

Reply

Claude is AI and can make mistakes. Opus 5.5 Medium Manual

2. I paste the question in Desktop.

3. I guess an answer, but ask Claude Desktop for help

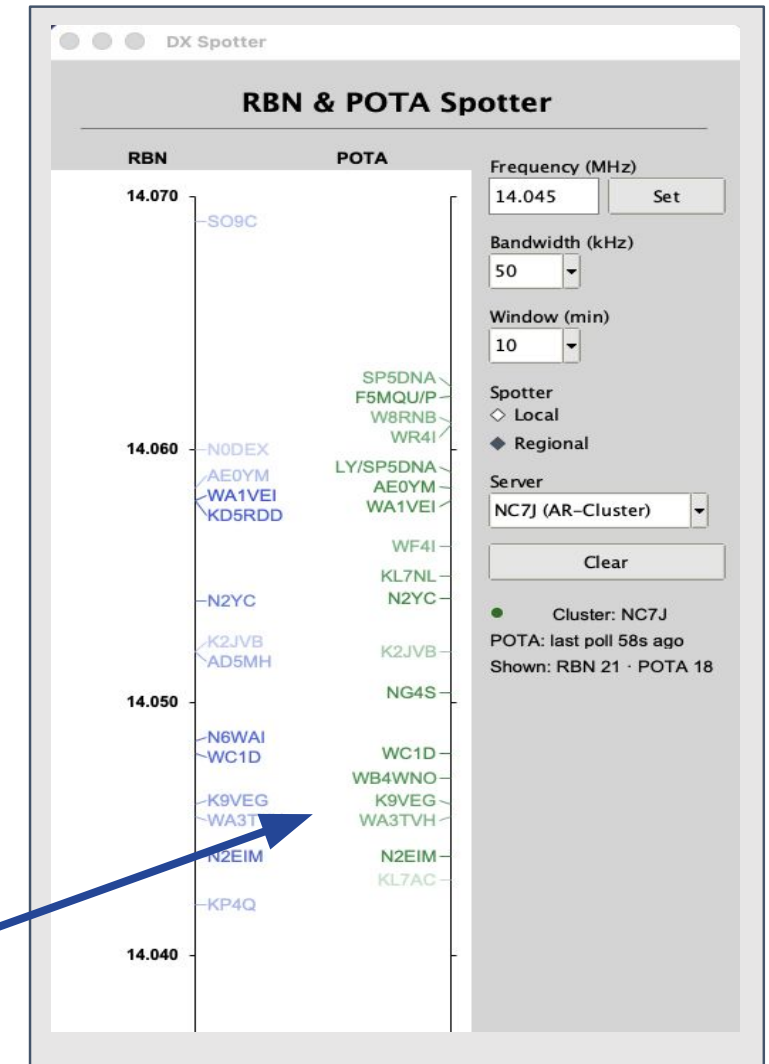
Claude Code

Claude Desktop

■ DX Spotter | The Working Result (Download from Link in Appendix)

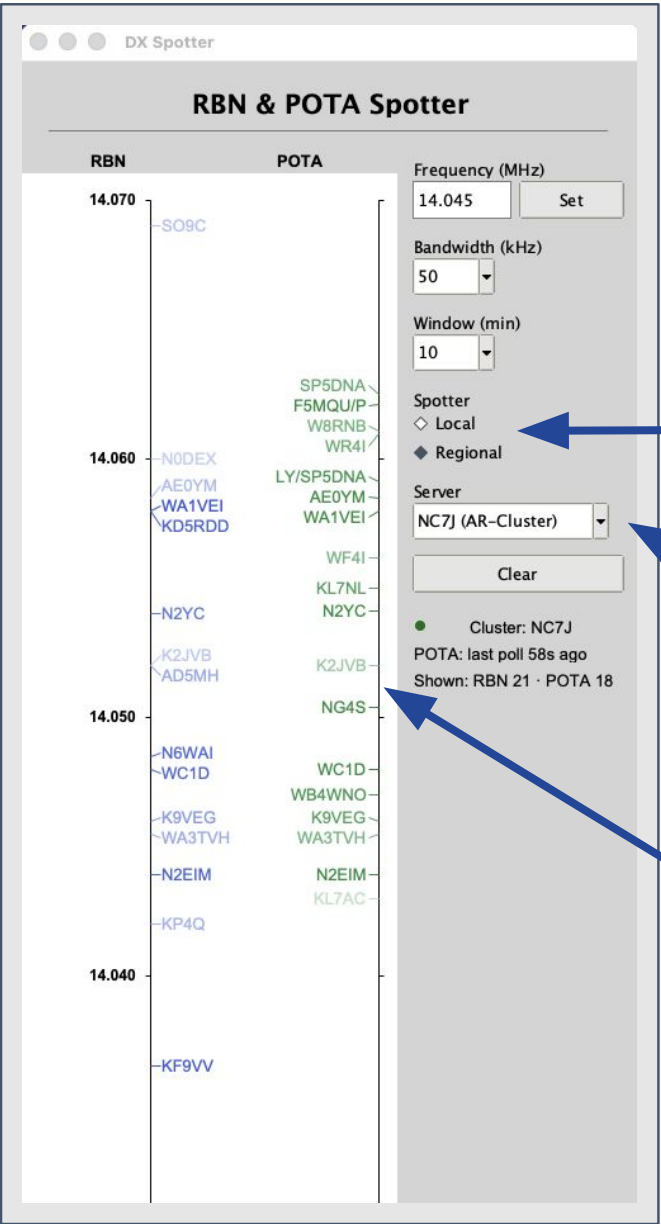
- Bandmap: live RBN spots, adjustable window: working (vs N1MM)
- POTA.APP hunting view: working (vs POTA.APP)
- Frequency, bandscope window width, timeout menu
- Spotter selection for Local (within ~50 miles) and Regional (within ~250 miles)
- Cluster server menu (3 major server types supported)
- Status indication for RBN & POTA servers
- Out of scope today: DE (Spotter) specification, CAT control, mobile UI

One build produced green spots for POTA so I kept that for starter screenshot



■ Multiplatform Screenshots (also runs on Linux)

Mac



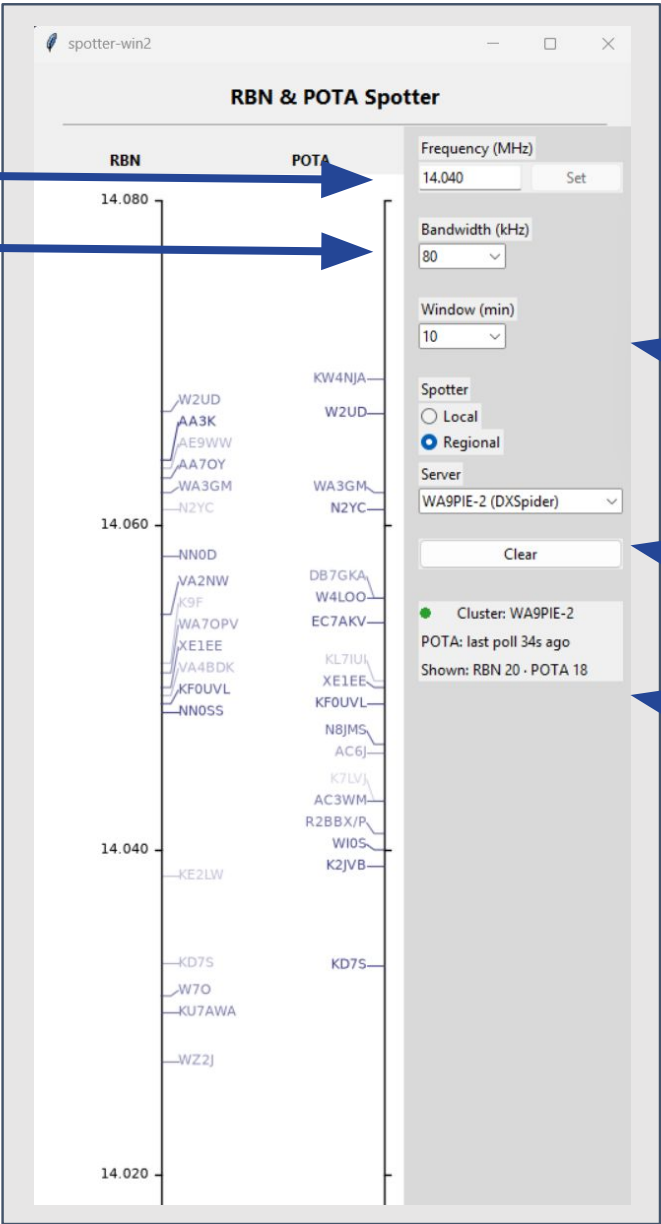
Center
Frequency
Bandwidth

Regional or
Local
Spotters

AR6
CC Cluster
DX Spider

Green POTA
Spots

Win



Window (min)

POTA Status

Spot Counts

Conclusion

Like a schematic, the spec is what you keep and share

Vibe coding gets you started. The Masterplan gets you finished.

1

Jot down the idea; then have agent build Masterplan...
...AND then have agent build code from it.

2

Build and test every step in task list, updating the
masterplan as you go, and GitHub remembering it.

3

Start small: Masterplan Generator with prompts, screenshot,
and your app idea.



The QR code points to the GitHub repo for this talk and
includes a Masterplan Generator file & Presentation

Your first masterplan
starts here

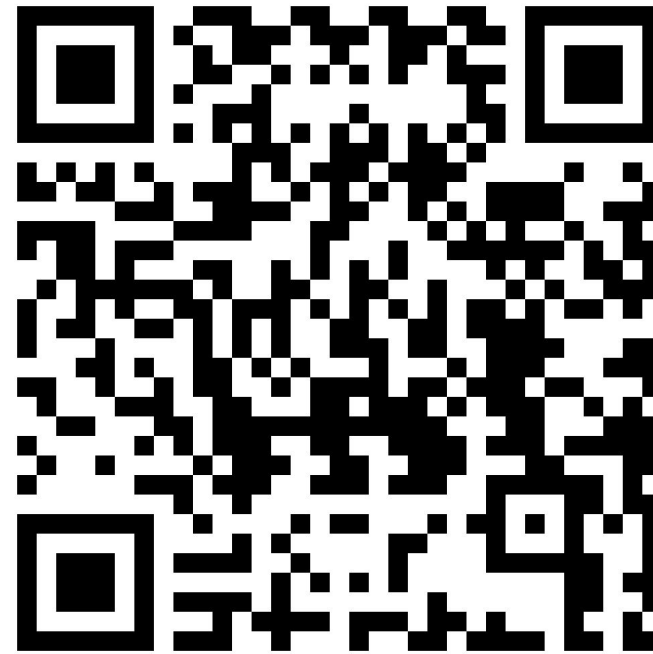
Appendix - GitHub Links



Talk repo

Slides, Example Masterplan,
Masterplan Generator

github.com/jcarter-labs/rsgb-sdd-n6yu



Code Repo

DX Spotter App Repo

github.com/jcarter-labs/dx-spotter-app

Contact

John Carter

N6YU

john@n6yu.com

github.com/jcarter-labs

■ Appendix - References

- "Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants", Deepak Babu Piskala: <https://arxiv.org/pdf/2602.00180>
- Claude Code Tutorials (Anthropic short courses): <https://anthropic.skilljar.com/>
- Field Spotter (Ian Renton): <https://git.ianrenton.com/ian/field-spotter>
- Deeplearning (Practical short course on application of SDD): [Spec-Driven Development with Coding Agents - DeepLearning.AI](#)
- Slides, masterplans, references: github.com/jcarter-labs/rsgb-sdd-n6yu

■ Find out more...

John Carter, N6YU

john@n6yu.com

github.com/jcarter-labs

www.rsgb.org

